UNIT V

JAVA FRAMEWORKS

MVC Architecture, Spring Framework, Spring Modules, Spring MVC and Spring Boot, Hibernate Framework, Other Popular Frameworks. Maven Installation, Maven Core Concepts - Basic Structure of the POM File, Maven Build Life Cycles, Phases and Goals - Maven Build Profiles

MVC ARCHITECTURE :

Model **V**iew **C**ontroller(MVC) is a design pattern which makes it easier to work on each part separately, so you can update or fix things without messing up the whole app. It helps developers add new features smoothly, makes testing simpler, and allows for better user interfaces. Overall, MVC helps keep everything organized and improves the quality of the software. MVC seperates the application into different objects. The main components of MVC pattern is

1. Model - This component is responsible for managing the application's data and processing business rules, and responding to requests for information from other components, such as the View and the Controller.
2. View - View displays the data from the Model to the user and sends user inputs to the Controller. It is passive and does not directly interact with the Model. Instead, it receives data from the Model and sends user inputs to the Controller for processing.
3. Controller - Controller acts as an intermediary between the Model and the View. It handles user input and updates the Model accordingly and updates the View to reflect changes in the Model. It contains application logic, such as input validation and data transformation.

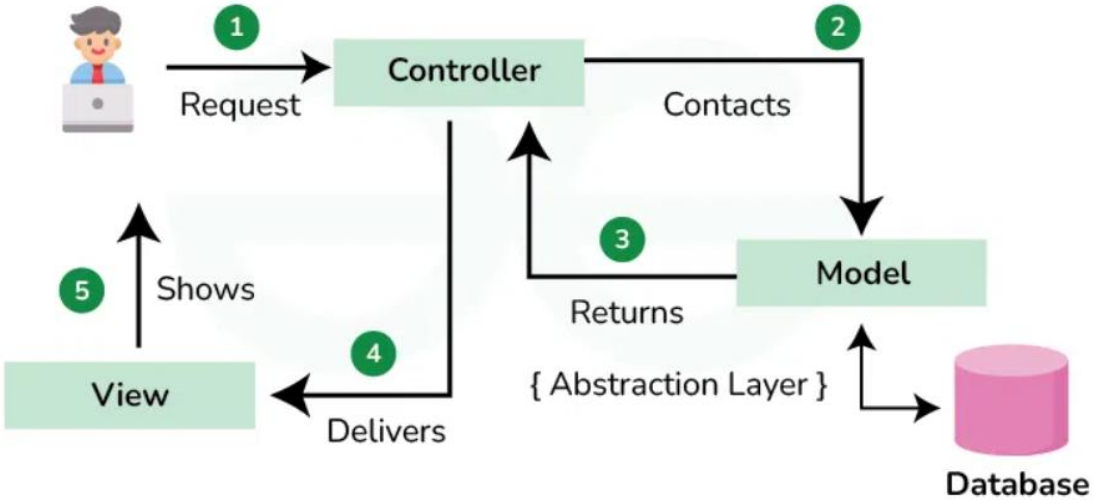


Fig 5.1 : MVC

Working of MVC :

STEP 1: **User Request -** The user sends the request by clicking a button or submitting a form or entering a URL

STEP 2: **Controller Layer to Model Layer -** The request goes to the controller and controller contacts the model and perfoms CRUD operations using HTTP methods (GET,POST,PUT,DELETE).

STEP 3: **Model to Controller -** The model layer interacts with the database and fetches the needed data and returns to the controller. (Note : This layer is an abstract layer, it doesn't know the details of database).

STEP 4: **Controller to View -** The controller delivers the data to the view layer.

STEP 5: **View to User -** View shows the final output to the user. It can be of any format like UI, webpage or a response.

INTRODUCTION TO MAVEN

Apache Maven is a build automation and project management tool used primarily for Java projects. It manages and simplifies project's compilation, building, testing, and dependency management using a standard project structure and configuration file. The main objectives of maven are

- Making the build process easy
- Providing a uniform build system
- Providing quality project information
- Encouraging better development practices

Maven uses an XML file for configuration called **pom.xml**. It is the heart of a maven project which defines project metadata, dependencies, build process, profiles and plugins. Some important elements in pom file are

- **groupId** → Organization / package name
- **artifactId** → Project name
- **version** → Project version
- **packaging** → jar / war / pom

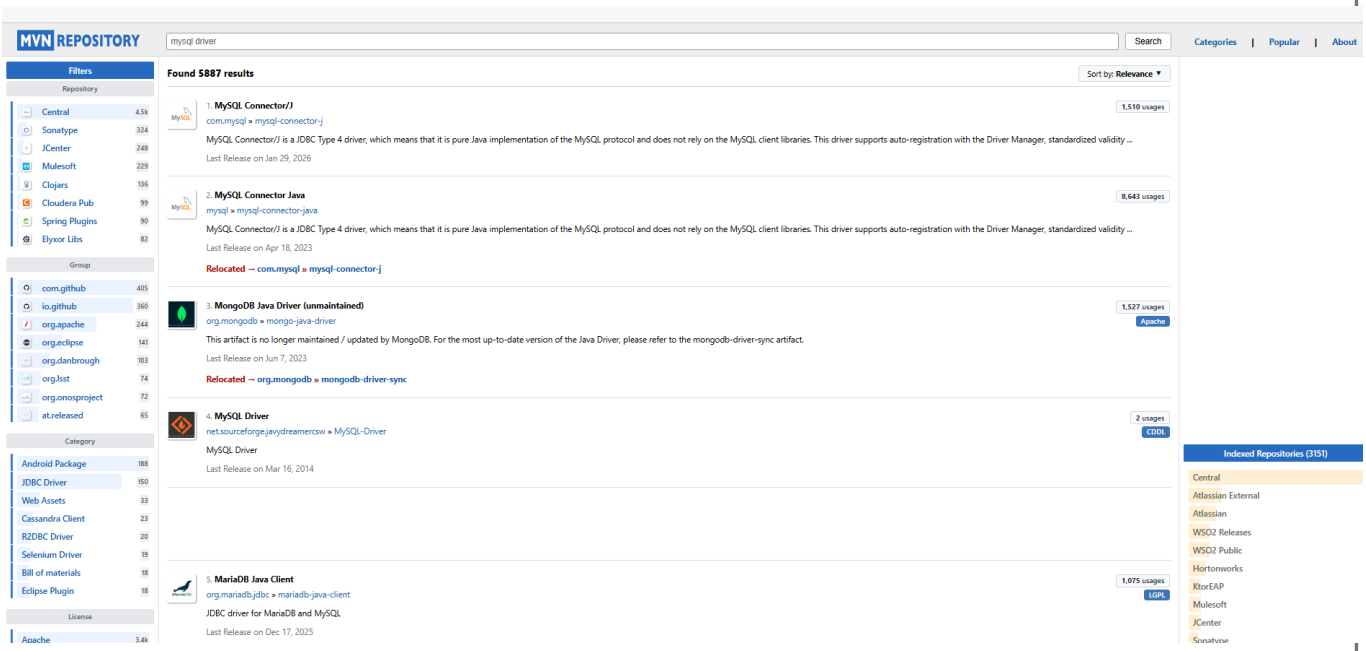
A simple pom.xml file looks like ,

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.demo</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0</version>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
```

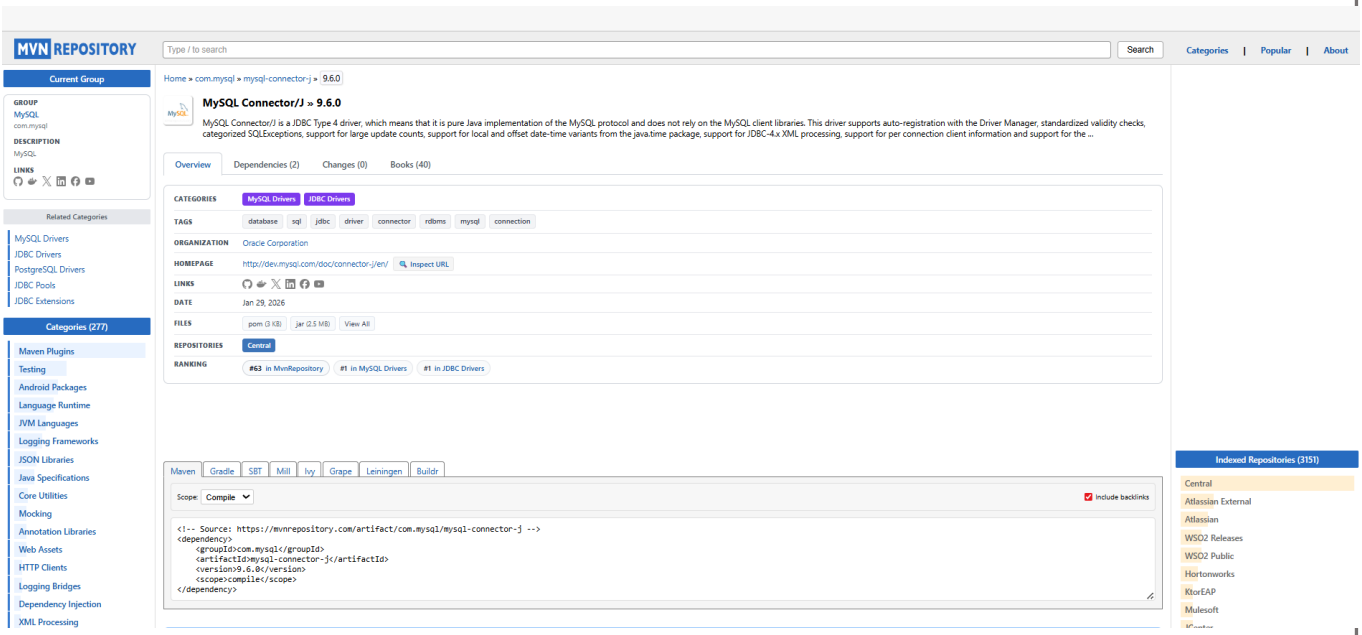
```
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
      </dependency>
    </dependencies>
</project>
```

We can manage dependencies by adding external dependancy from external sites like mvnrepository or maven central repository.

STEP 1:



STEP 2:



Some example dependancy are

MySQL JDBC driver.

Add this dependancy in < dependencies > tag in **pom.xml** file

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.4.0</version>
</dependency>
```

MAVEN PHASES AND BUILD LIFECYLCES

A lifecycle is an ordered sequence of phases.

Common phases involved in lifecycle:

- validate – check project structure and config

- compile – compile source code
- test – run unit tests
- package – create the artifact (JAR/WAR/etc.)
- verify – run integration checks
- install – install artifact to local repo (~/.m2)
- deploy – publish artifact to a remote repo

  This is the most commonly used lifecycle to build the application.
  - validate → compile → test → package → verify → install → deploy

How to install Maven

STEP 1 : Download maven zip file from https://maven.apache.org/download.cgi

STEP 2 : Extract it to C: drive and note the full path.

STEP 3 : Search for "Edit the system environment variables" in start menu and open it.

STEP 4 : Under System variables, click New: Variable name MAVEN_HOME, Value is the Maven folder path (e.g., C:\Program Files\Apache\Maven\apache-maven-3.9.x). Click OK.

STEP 5 : In Environment Variables, select Path under System variables and click Edit. Click new and add %MAVEN_HOME%\bin. Click OK on all windows.

STEP 6 : Verify installation by running mvn -version command in command prompt.

BUILD PROFILES

A Maven build profile is a named set of configuration changes that can be conditionally applied during a build. Profiles let the same project be built in different ways without modifying the POM each time. Normally it uses single effective POM. When a profile is activated, Maven merges that profile's configuration into the effective POM for that build only.

It is mainly used in Environment-specific resource filtering(dev, test, prod etc.,), Different properties (DB URLs, API endpoints). Profiles can be defined in pom.xml.

Sample profile:

1. dev- profile

<profiles>

```
 <profile>
   <id>dev</id>
   <properties>
     <env>dev</env>
   </properties>
 </profile>
```

2.prod- profile

```
<profile>
   <id>prod</id>
   <properties>
     <env>prod</env>
   </properties>
 </profile>
</profiles>
```

```
mvn clean package -Pdev
mvn clean package -Pprod
```

ANNOTATION

Annotation is a special kind of metadata that you add to code to give extra information to the compiler, tools, or frameworks. Annotations do not change program logic directly, but they influence how the code is processed or behaves at runtime. For eg.,

```
@Override
public String toString() {
```

```
        return "Hello";

    }
```

Here @Override is an annotation which provides an information that the method toString() is overrided.

SPRING FRAMEWORK

Spring is a popular open source framework created to address the complexity of enterprise application development. It is a lightweight inversion of control and AOP framework. It's mainly built using Inversion of Control and Dependancy Injection.

There are many sub projects in spring called spring modules. Some of the projects are SpringBoot, Spring Security, Spring Core, Spring DataJPA, Spring Cloud etc., At its core, Spring offers a **container**, often referred to as the Spring application context, that creates and manages application components. These components, or beans, are wired together inside the Spring application context to make a complete application, much like bricks, mortar, timber, nails, plumbing, and wiring are bound together to make a house. The act of wiring beans together is based on a pattern known as **dependency injection** (DI). Rather than have components create and maintain the lifecycle of other beans that they depend on, a dependency-injected application relies on a separate entity (the container) to create and maintain all components and inject those into the beans that need them. This is done typically through constructor arguments or property accessor methods.

How to create a spring project?

1. Download and install maven or any build tool specific to java like Maven, Gradle, Ant etc.
2. Add spring-core, spring-context dependencies into the <dependencies></ dependencies> tag.
3. Select maven clean and install to add the dependancies or reload the maven to add the dependancy files.
4. Spring can be configured in 3 ways
    - **XML-based configuration** - Spring xml file is create as file and the below xml tags are added to that file

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xmlns:context="http://www.springframework.org/schema/context"

xsi:schemaLocation="http://www.springframework.org/schema/beans

https://www.springframework.org/schema/beans/spring-beans.xsd

http://www.springframework.org/schema/context

https://www.springframework.org/schema/context/spring-context.xsd">

<bean name="helloworld" class= "com.dev.helloworld">

</beans>

- **Java-based configuration** - Java-based configuration option enables you to write most of your Spring configuration without XML but with the help of few Java-based annotations like @Configuration, @Bean.

  Eg.,

```
@Configuration
public class HelloWorldConfig
{
   @Bean
   public HelloWorld helloWorld()
   {
      return new HelloWorld();
   }
}
```

- **Annotation based configuration** - Annotation-based configuration in Spring means configuring your application using annotations instead of XML. It makes the code cleaner, easier to read, and easier to maintain.

  Eg.,

  @Component is annotated to a class to make the class as bean

```
@Component
public class HelloWorldConfig
{
   public static void display()
   {
    System.out.println("Display message called…")
   }
}
```

SPRING BEANS

A Spring Bean is an object that is created, managed, and destroyed by the Spring IoC container. It is created by Spring container to promote loose coupling and automatic dependancy injection.

Spring bean can be created in 2 ways , one is in XML file by using

```
<bean id="userService" class="com.example.UserService"/>
```

Next is using annotations, @Component is the most common way to create bean for a class.

```
@Component
public class UserService {
}
```

Some specialized bean annotations are @Service, @Repository, @Controller. @Bean is used to create bean for a class in @Configuration class.

IoC & DEPENDANCY INJECTION

IoC container has the control of creating objects and managing dependencies. In simple terms, Instead of you creating objects using **new** keyword, Spring does it for you. It can be created using ApplicationContext and BeanFactory interfaces.

ApplicationContext context = new ClassPathXmlApplicationContext("Spring.xml");

This line creates a container and add the bean configured in the xml.

```
<bean id="SpringApp"  class="com.dev.SpringApp ">
        <!-- additional collaborators and configuration for this bean go here -->
</bean>
```

Dependency Injection is a design pattern where a class receives its required objects (dependencies) from an external source (an "injector") rather than creating them itself, promoting loose coupling, testability, and flexibility by separating object creation from object usage.

Dependancy Injection can be achieved in 3 ways,

- **Constructor-Injection** - Dependencies are passed through the class's constructor
- **Property/Setter Injection** - Dependencies are set via public properties or setter methods after the object is created.
- **Field Injection** - Dependencies are passed directly to the method that needs them using annotations.

Example :

Note : This code works after configuring all the neccesary dependancies for Spring.

SPRING MODULES

1.Core Container

This is the backbone of the Spring Framework and manages object creation and dependency injection. It provides the IoC (Inversion of Control) container that wires application components together. All other Spring modules depend on this container. It also supports configuration using annotations and XML.

- **spring-core**: Provides core utilities and fundamental IoC functionality used throughout Spring.
- **spring-beans**: Manages bean creation, configuration, dependency injection, and lifecycle.
- **spring-context**: Builds on core and beans to provide ApplicationContext, event handling, and resource loading.

2. Data Access / Integration

This module simplifies interaction with databases and enterprise systems. It removes boilerplate code and provides consistent exception handling. It also supports transactions and ORM frameworks. Developers can switch data access technologies easily.

- **spring-jdbc**: Simplifies JDBC operations and handles resource management.
- **spring-tx**: Provides declarative and programmatic transaction management.

- **spring-orm**: Integrates ORM tools like Hibernate and JPA with Spring.

3. Web

This module is used to build web applications and RESTful services. It supports both traditional servlet-based and reactive programming models. Spring handles request routing, data binding, and response generation.

- **spring-web**: Provides core web features, REST support, and HTTP utilities.
- **spring-webmvc**: Implements the MVC architecture using controllers, views, and models.
- **spring-webflux**: Supports reactive, non-blocking web applications.

4. Security

Spring Security protects applications from unauthorized access. It handles authentication, authorization, and secure communication. It integrates easily with web, REST, and microservices.

- **Spring Security**: Supports login, role-based access, OAuth2, JWT, and CSRF protection.

5.Testing

This module supports unit and integration testing of Spring applications. It allows loading Spring context during tests. It works well with popular testing frameworks.

- **spring-test**: Provides testing support for JUnit, Mockito, and integration tests.

Spring also has other modules like AOP, Instrumentation, Messaging, and more, which are used as per specific project requirements.

SPRING BOOT

Spring Boot is not a replacement for Spring MVC, but a framework to simplify Spring applications. It makes it easier to build standalone, production-ready Spring apps with minimal configuration.

Key features of Spring boot are

- **Auto-configuration** – It automatically configures spring components based on the dependancies in the project
- **Embedded Server** - It comes with embedded Tomcat so you don't need to deploy WAR files to an external server.
- **Starter Dependencies** - It has predefined dependency sets for common tasks. Eg: spring-boot-starter-web for web, spring-boot-starter-data-jpa for database, etc.
- **Minimal Configuration** - Reduces XML and manual annotation configuration. just a main class with @SpringBootApplication is enough to start a web server and run a Spring MVC app.

A simple SpringBoot application

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Server configurations are added in application.properties file which is commonly used in Springboot applications which stores application settings such as database details, server ports, logging levels, and more. It is located in src/main/resources by default. It works in key-value pair.

Property	Purpose	Example
server.port	Set server port	server.port=8081
spring.datasource.url	Database URL	spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username	DB username	spring.datasource.username=root
spring.datasource.password	DB password	spring.datasource.password=pass123
spring.jpa.hibernate.ddl-auto	JPA schema handling	spring.jpa.hibernate.ddl-auto=update

SPRING MVC

Spring MVC (Model-View-Controller) is a framework for building web applications in Java. It is part of the Spring Framework. It uses MVC pattern where **model** represents java objects, database records, **view** represents the user interface like JSP,HTML etc., controller handles user requests and returns a response.

Annotation	Purpose
@Controller	Marks a class as a Spring MVC controller. It is used for handling web requests and returning views.
@RequestMapping	Maps HTTP requests to controller methods. Can specify URL patterns, HTTP methods, etc.

@GetMapping, @PostMapping, @PutMapping, @DeleteMapping	Shortcut annotations for @RequestMapping for specific HTTP methods.
@RestController	Combines @Controller and @ResponseBody. Used for building REST APIs where methods return JSON or XML.
@ResponseBody	Indicates that the return value of a method should be written directly to the HTTP response body (used in REST APIs).
@Service	Marks a class as a service layer component, containing business logic.
@Repository	Marks a class as a DAO (data access object) component. Handles database operations.

REST API

REST (Representational State Transfer) is an architectural style for designing web services. A REST API allows clients (like web browsers, mobile apps, or other services) to communicate with your server using HTTP methods GET, PUT, POST, DELETE. Springboot makes creating REST APIs very simple with annotations like @RestController and @RequestMapping.

First we need to add the dependancies for springboot spring-boot-starter-web, spring-boot-starter-test.

A sample Student management code

```
//CONTROLLER

package com.example.studentcrud.controller;


import com.example.studentcrud.model.Student;

import com.example.studentcrud.service.StudentService;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping("/students")

public class StudentController {


    private final StudentService studentService;


    public StudentController(StudentService studentService) {

        this.studentService = studentService;

    }


    // Create

    @PostMapping

    public Student addStudent(@RequestBody Student student) {
```

```
        return studentService.addStudent(student);
    }

    // Read all
    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    // Read by id
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable int id) {
        return studentService.getStudentById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // Update
    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(@PathVariable int id, @RequestBody
Student student) {
        return studentService.updateStudent(id, student)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

```
    // Delete
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(@PathVariable int id) {
        if (studentService.deleteStudent(id)) {
            return ResponseEntity.ok().build();
        } else {
            return ResponseEntity.notFound().build();
        }
    }
}

//SERVICE
package com.example.studentcrud.service;

import com.example.studentcrud.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class StudentService {
```

```
    private final List<Student> students = new ArrayList<>();
    private int currentId = 1;

    // Create student
    public Student addStudent(Student student) {
        student.setId(currentId++);
        students.add(student);
        return student;
    }

    // Get all students
    public List<Student> getAllStudents() {
        return students;
    }

    // Get student by ID
    public Optional<Student> getStudentById(int id) {
        return students.stream().filter(s -> s.getId() == id).findFirst();
    }

    // Update student
    public Optional<Student> updateStudent(int id, Student updatedStudent) {
        return getStudentById(id).map(student -> {
```

```
            student.setName(updatedStudent.getName());
            student.setEmail(updatedStudent.getEmail());
            return student;
        });
    }

    // Delete student
    public boolean deleteStudent(int id) {
        return students.removeIf(student -> student.getId() == id);
    }
}
//MODEL
package com.example.studentcrud.model;

public class Student {
    private int id;
    private String name;
    private String email;

    public Student() {}

    public Student(int id, String name, String email) {
        this.id = id;
        this.name = name;
```

```
    this.email = email;
  }

  public int getId() {
    return id;
  }

  public void setId(int id) {
    this.id = id;
  }

  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public String getEmail() {
    return email;
  }

  public void setEmail(String email) {
```

```
        this.email = email;
    }
}
```

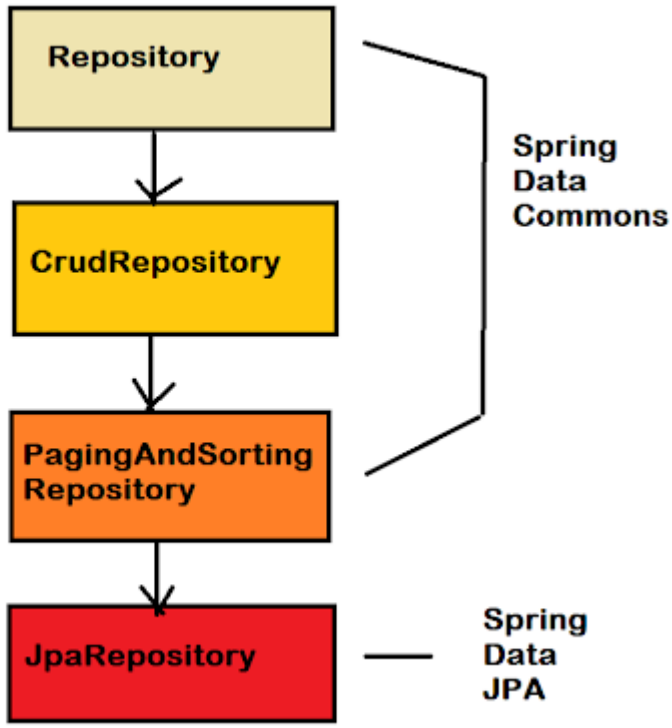
HIBERNATE FRAMEWORK

Hibernate is a Java ORM (Object-Relational Mapping) framework. It maps Java objects to database tables and handles SQL automatically, so you don't have to write raw SQL queries for common operations.

Key features of Hibernate are

- Automatic mapping between Java classes and DB tables
- Transparent persistence (CRUD operations)
- HQL (Hibernate Query Language) for database queries
- Caching, lazy loading, and transaction management

Spring Boot simplifies Spring applications by auto-configuring things. Hibernate integrates seamlessly with Spring Boot, usually via **Spring Data JPA**, which is a Spring abstraction on top of Hibernate.



A sample Student registration code

```
//CONTROLLER
import com.dev.studentregistration.model.Student;
import com.dev.studentregistration.service.HomeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class HomeController {

    @Autowired
    private HomeService homeService;

    @GetMapping()
    public ResponseEntity<List<Student>>  getStudents() {
        return homeService.getStudents();
    }

    @GetMapping("/filter")
    public ResponseEntity<List<Student>> getStudentbyDept(@RequestParam("d") String
dept){
        return homeService.getStudentByDept(dept);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentbyId(@PathVariable("id") int id){
        return homeService.getStudentByID(id);
    }
```

```
    @PostMapping
    public String createStudent(@RequestBody Student student){
        return homeService.updateStudentData(student);
    }

    @PutMapping
    public String UpdateStudent(@RequestBody Student student){
        return homeService.updateStudentData(student);
    }


}
```

//SERVICE

```
import com.dev.studentregistration.model.Student;
import com.dev.studentregistration.repo.Repo;
import org.apache.coyote.Response;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class HomeService {

    private Repo repo;

    @Autowired
    public HomeService(Repo repo) {
        this.repo = repo;
```

```
    }

    public ResponseEntity<List<Student>> getStudents() {
        List<Student> std = repo.findAll();
        if(std.isEmpty()){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(std);
    }



    public ResponseEntity<List<Student>> getStudentByDept(String department) {
//        List<Student> std = repo.findAll().stream().filter(d ->
d.getDepartment().equals(department)).toList();
//        if(std.isEmpty()){
//            return ResponseEntity.notFound().build();
//        }

        List<Student> std = repo.findBydepartment(department);
        if(std.isEmpty()){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(std);
    }

    public ResponseEntity<Student> getStudentByID(int id) {
        var student = repo.findById(id).orElse(null);
        if(student == null){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(student);
    }

    public String updateStudentData(Student student) {
```

```
        String result = "";
        try {
            repo.save(student);
            result = "Student Created Successfully";
        }
        catch (Exception e) {
            ResponseEntity.badRequest().build();
        }
        return result;
    }
}
```

//REPOSITORY

```
package com.dev.studentregistration.repo;

import com.dev.studentregistration.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface Repo extends JpaRepository<Student,Integer> {
    List<Student> findBydepartment(String department);
}
```

//MODEL

```
package com.dev.studentregistration.model;

import jakarta.persistence.*;

@Entity
@Table(name = "admission")
public class Student {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int sid;
private String name;
private String address;
private String email;

@Column(name="phoneNo")
private long phoneNo;

public Student() {

}

public int getSid() {
   return sid;
}

public void setSid(int sid) {
   this.sid = sid;
}

public String getName() {
   return name;
}

public void setName(String name) {
   this.name = name;
}

public String getAddress() {
   return address;
}
```

```
    public void setAddress(String address) {
        this.address = address;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public long getPhoneNo() {
        return phoneNo;
    }

    public void setPhoneNo(long phoneNo) {
        this.phoneNo = phoneNo;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }


    public Student(int sid, String name, String address, String email, long phoneNo, String
department) {
        this.sid = sid;
        this.name = name;
```

```
        this.address = address;
        this.email = email;
        this.phoneNo = phoneNo;
        this.department = department;
    }

    private String department;
}
```

//MAIN

```
package com.dev.studentregistration;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentRegistrationApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentRegistrationApplication.class, args);
    }

}
```

OTHER POPULAR FRAMEWORKS

1. Micronaut

Micronaut is a lightweight, modern JVM framework designed for building microservices and cloud-native applications. It emphasizes fast startup time and low memory consumption by using compile-time dependency injection instead of reflection. Micronaut integrates well with cloud platforms, supports REST APIs, and works smoothly with GraalVM for native image generation, making it ideal for serverless and container-based deployments.

2.Quarkus

Quarkus is a Kubernetes-native Java framework optimized for containers and microservices. It offers extremely fast boot times and low memory usage, making it suitable for cloud environments. Quarkus supports both imperative and reactive programming models and integrates with popular technologies like RESTEasy, Kafka, and various persistence libraries. It is particularly popular for building scalable applications in modern DevOps ecosystems.

3.Dropwizard

Dropwizard is a simple yet powerful framework for developing RESTful web services. It bundles together stable and mature libraries such as Jetty for HTTP, Jersey for REST, and Jackson for JSON processing, allowing developers to quickly create production-ready applications. Dropwizard focuses on simplicity, performance, and operational readiness, making it a strong choice for building reliable backend services.

4.Play

Play Framework is a high-performance, reactive web framework that supports both Java and Scala. It follows the MVC architecture and promotes asynchronous, non-blocking programming to handle high traffic efficiently. Play Framework is developer-friendly, supports hot reloading during development, and is often used to build scalable and modern web applications with real-time features.

5.Vert.x

Vert.x is an event-driven, non-blocking toolkit for building reactive and distributed systems on the JVM. Unlike traditional frameworks, Vert.x provides a flexible toolkit approach rather than a strict application structure, giving developers more control. It supports polyglot programming (Java, Kotlin, Groovy, etc.) and is well-suited for applications requiring high concurrency, such as real-time messaging systems and streaming platforms.